

Chương 12: Mô hình tùy chỉnh và Huấn luyện với TensorFlow

Giảng viên: TS. Trần Thành Thắng

Đại học Đông Á

Năm học 2024-2025

Nội dung chương

- 1 Giới thiệu TensorFlow & Kiến trúc
- 2 Sử dụng TensorFlow như NumPy
- 3 Tùy chỉnh mô hình và Thành phần (Custom Components)
- 4 Hàm và biểu đồ TensorFlow (AutoGraph)
- 5 Vòng lặp Huấn luyện Tùy chỉnh

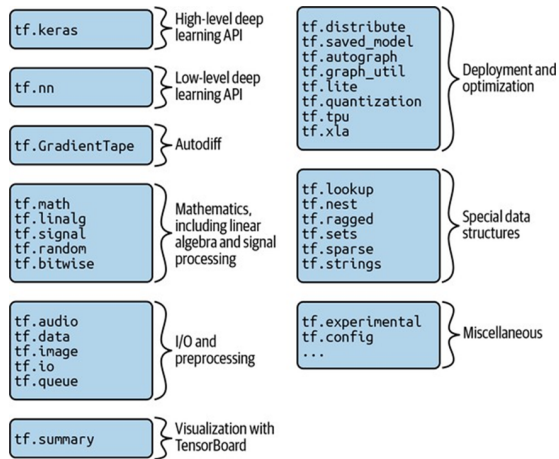
Giới thiệu nhanh về TensorFlow

- TensorFlow là thư viện tính toán số học mạnh mẽ mã nguồn mở, đặc biệt phù hợp cho học máy quy mô lớn và học sâu.
- Cấu trúc của nó tương tự NumPy, nhưng thao tác với **Tensor** thay vì mảng.
- Cung cấp khả năng tính toán vượt trội nhờ hỗ trợ phần cứng tăng tốc: **GPU** (Graphics Processing Unit) và **TPU** (Tensor Processing Unit).
- Tính năng cốt lõi: Tính toán phân tán trên hàng ngàn thiết bị và server.
- Có thể biên dịch tự động các biểu thức tính toán Python thành **Biểu đồ (Graph)** C++ hiệu suất cao thông qua công nghệ AutoGraph.

- **TensorBoard**: Trực quan hóa dữ liệu, đồ thị mô hình và phân tích quá trình huấn luyện.
- **TensorFlow Hub**: Nơi tải xuống và sử dụng lại các mô hình đã huấn luyện (Pretrained models) giúp đẩy nhanh tiến độ dự án.
- **TensorFlow Extended (TFX)**: Bộ công cụ chuyên biệt để xây dựng quy trình ML toàn diện và triển khai mô hình lên môi trường sản xuất (Production).
- **TF Lite & TF.js**: Hỗ trợ triển khai mô hình học máy trên thiết bị di động (Android, iOS), phần cứng nhúng (Raspberry Pi) và ngay trên trình duyệt Web bằng JavaScript.

Tổng quan về API Python của TensorFlow (1/2)

- API Python của TensorFlow được thiết kế phân tầng từ thấp đến cao.
- Ở mức thấp, ta có các API nền tảng: các phép toán trên Tensor, biến số (Variables), tự động vi phân (tf.GradientTape).
- Ở cấp độ cao hơn (Cấp cao), Keras cung cấp API trực quan (tf.keras) để xây dựng lớp (layers) và mô hình (models).
- API dữ liệu (tf.data) cung cấp quy trình nạp, xử lý trước và cung cấp dữ liệu huấn luyện hiệu quả.



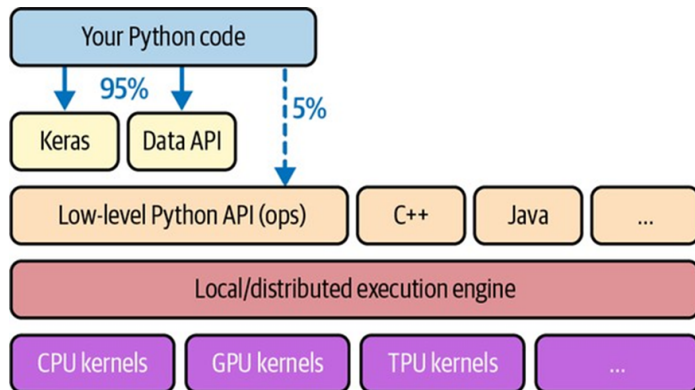
Hình 12-1: Tổng quan về API Python của

Tổng quan về API Python của TensorFlow (2/2)

- Sự trừu tượng hóa các mức độ phức tạp giúp người học mới tiếp cận qua Keras một cách nhanh chóng.
- Trong khi đó, các nhà nghiên cứu sâu (researchers) vẫn có thể dùng các API cấp thấp để viết các thuật toán huấn luyện phi tiêu chuẩn (như trong GANs hay Meta-learning).
- Hình 12-1 tóm tắt một cách bao quát các module quan trọng mà TensorFlow cung cấp.
- Chúng ta sẽ lần lượt khám phá từ Tensor cơ bản đến việc tùy chỉnh các lớp và mô hình thông qua API này.

Kiến trúc của TensorFlow (1/2)

- Hình 12-2 minh họa kiến trúc bên trong của TensorFlow.
- Lớp Execution Engine ở trung tâm thực hiện các biểu đồ (Graphs) bằng C++ (mã nguồn lỗi được tối ưu cực cao).
- Cung cấp khả năng chạy đồng thời (multi-threading) trên nhiều loại phần cứng (CPU, GPU, TPU) và qua mạng (Android, iOS, Cloud).



Hình 12-2: Kiến trúc của TensorFlow

Kiến trúc của TensorFlow (2/2)

- API giao diện bằng nhiều ngôn ngữ: Python (phổ biến nhất), C++, Java, Go, Swift.
- Hầu hết thời gian bạn sẽ chỉ cần tương tác qua Keras (High-level API).
- Lớp **XLA (Accelerated Linear Algebra)**: Trình biên dịch mạnh mẽ, giúp tối ưu hóa thêm các Graph, kết hợp các toán tử lại (kernel fusion) nhằm tăng tốc tối đa tốc độ xử lý phần cứng.
- TF Distributed Execution: Trực tiếp phân phối khối lượng tính toán trên nhiều máy vật lý (Parameter Server, MirroredStrategy).

Tensor và Phép toán (Tensors and Operations)

- API của TensorFlow chủ yếu xoay quanh các *tensors* (cấp số liệu đa chiều).
- Khái niệm Tensors được kế thừa từ Toán học, dùng để đại diện cho đại lượng vô hướng (scalar - 0D), vectơ (1D), ma trận (2D) hoặc không gian đa chiều (nD).
- Cách dễ nhất để tạo tensor là dùng `tf.constant()`.

```
import tensorflow as tf

# Tao tensor tu ma tran 2 chieu (2D)
t = tf.constant([[1., 2., 3.], [4., 5., 6.]])
print(t.shape) # In ra: TensorShape([2, 3])
print(t.dtype) # In ra: <dtype: 'float32'>
```

Các hàm toán học cơ bản

- TensorFlow cung cấp hàng loạt hàm toán học mạnh mẽ hoạt động trực tiếp trên tensors mà không cần vòng lặp for.
- Các toán tử chuẩn Python (+, -, *, /, **) đã được nạp chồng (overloaded).

```
# Tính toán cơ bản bằng TF operations
t1 = t + 10                # Phép cộng (hoạt động broadcasting)
t2 = tf.square(t)          # Bình phương từng phần tử (element-wise)
t3 = t @ tf.transpose(t)   # Nhân ma trận (Matrix multiplication)
```

- Ký hiệu @ xuất hiện từ Python 3.5 để thay thế cho phép nhân ma trận (tf.matmul).
- Các hàm phổ biến khác: tf.exp(), tf.sqrt(), tf.reduce_sum(), tf.reduce_mean().

Tensors và NumPy: Tương tác qua lại

- Tensors tương tác rất thân thiện và liền mạch với thư viện NumPy - tiêu chuẩn của khoa học dữ liệu.
- Có thể tạo tensor trực tiếp từ một mảng NumPy, và ngược lại.

```
import numpy as np

# Chuyển Numpy -> Tensor
a = np.array([2., 4., 5.])
t_a = tf.constant(a)

# Chuyển Tensor -> Numpy
a_back = t_a.numpy()           # Ban có thu được np.array(t_a)

# Tính toán kết hợp (Cross-computation)
t_b = tf.square(a)             # TF chấp nhận Numpy array
a_b = np.square(t_a)           # Numpy chấp nhận TF Tensor
```

Lưu ý khi dùng chung Numpy và TensorFlow

- Mặc dù chúng có thể được truyền vào hàm của nhau, nhưng bạn nên ưu tiên dùng hàm TensorFlow nếu muốn tận dụng tốc độ của GPU.
- NumPy chỉ chạy được duy nhất trên CPU. Nếu một Tensor đang nằm trên GPU mà bạn gọi hàm NumPy, TensorFlow sẽ phải **sao chép dữ liệu từ bộ nhớ GPU về CPU** → Chi phí truyền tải lớn.
- Khác biệt nhỏ về kiểu dữ liệu: NumPy mặc định dùng float64 cho độ chính xác cao. TensorFlow mặc định dùng float32 (thậm chí bfloat16) vì mạng Neural quan trọng tính hiệu suất.

Chuyển đổi kiểu dữ liệu (Type Conversions)

- TensorFlow **không tự động** ép kiểu (type conversion) giống như Python hay NumPy.
- Việc thực hiện ép kiểu giữa float32 và float64 hoặc float32 và int32 có chi phí tính toán cao.

```
# Kiểu int32 không thể cộng với float32 ngay dinh:  
# tf.constant(2.0) + tf.constant(40) # --> Lỗi TypeError  
  
# Cần phải ép kiểu chu dòng và rõ ràng bằng tf.cast:  
t2 = tf.constant(40., dtype=tf.float64)  
  
# tf.cast(t2, tf.float32) hạ độ chính xác xuống float32  
result = tf.constant(2.0) + tf.cast(t2, tf.float32)
```

Biến số (Variables)

- Đặc tính của `tf.constant()` là không thể thay đổi (immutable). Ta không thể gán lại một giá trị cho một phần tử bên trong nó.
- Tuy nhiên, mạng nơ-ron liên tục cập nhật **trọng số (weights)** và độ lệch (biases) qua mỗi chu kỳ học (epoch).
- Do đó, ta cần sử dụng cấu trúc `tf.Variable`.

```
v = tf.Variable([[1., 2., 3.], [4., 5., 6.]])  
  
# Cap nhat toan bo bien (dung phuong thuc assign)  
v.assign(2 * v)  
  
# Cap nhat vao dung 1 vi tri cu the (nhu numpy arrays)  
v[0, 1].assign(42)  
  
# Cap nhat toan cuc bang phap cong (+), tru (-)  
v.assign_add([[1., 1., 1.], [1., 1., 1.]])
```

Các cấu trúc dữ liệu nâng cao khác

- **Sparse tensors** (`tf.SparseTensor`): Tối ưu bộ nhớ cho ma trận chứa phần lớn giá trị là 0. Lưu trữ 3 mảng: *indices*, *values*, *dense_shape*.
- **Tensor arrays** (`tf.TensorArray`): Một danh sách các Tensors cho phép append động. Rất hữu ích cho các mạng tuần tự RNN.
- **Ragged tensors** (`tf.RaggedTensor`): Đại diện mảng bất đối xứng. VD: danh sách chứa các chuỗi con có độ dài không bằng nhau.
- **String tensors**: Tensor lưu trữ kiểu chữ (strings), các chuỗi ký tự UTF-8 hoặc byte arrays, cần thiết cho NLP.
- **Sets**: Tập hợp Toán học dưới dạng Sparse tensor bằng `tf.sets`.

Tại sao cần tùy chỉnh mô hình trong Keras?

- Keras API cung cấp sẵn hàng chục hàm *losses*, *metrics*, *layers*, *optimizers* chuẩn mực. Đa số bài toán chỉ cần gọi tên chuỗi, ví dụ: `loss="mse"`, `activation="relu"`.
- **Thực tế phức tạp hơn:**
 - ➊ Bài toán đặc thù yêu cầu hàm tính sai số hoàn toàn mới: Phạt lỗi sai số dương nặng gấp đôi sai số âm.
 - ➋ Nghiên cứu tạo ra một cấu trúc Tầng Nơ-ron (Layer) chưa từng có.
 - ➌ Xây dựng mạng với 3 nguồn dữ liệu vào (Multi-input) và 2 luồng kết quả xuất ra (Multi-output).
- TensorFlow 2 cho phép dùng Python viết mọi thứ, đóng gói lại và dùng như Module nguyên bản.

Tùy chỉnh Hàm mất mát (Custom Loss) - Huber Loss (1/2)

- Ví dụ bài toán dự đoán giá cả. Lỗi ($\text{error} = \text{label} - \text{prediction}$).
- Mean Squared Error (MSE) phạt bình phương lỗi, rất nhạy cảm nhiễu. Mean Absolute Error (MAE) trừng phạt tuyến tính, nhưng không mượt quanh điểm 0.
- **Huber Loss:** Hành vi giống MSE khi lỗi nhỏ, giống MAE khi lỗi lớn.

```
def huber_fn(y_true, y_pred):  
    error = y_true - y_pred  
    is_small_error = tf.abs(error) < 1  
    squared_loss = tf.square(error) / 2  
    linear_loss = tf.abs(error) - 0.5  
    # Dùng tf.where() giống như np.where()  
    return tf.where(is_small_error, squared_loss, linear_loss)
```

Tùy chỉnh Hàm mất mát (Custom Loss) - Sử dụng (2/2)

- Dùng hàm Python trực tiếp vào tham số loss khi biên dịch mô hình.

```
# Truyền trực tiếp tên hàm huber vào phương thức compile
model.compile(loss=huber_fn, optimizer="nadam", metrics=["mae"])

# Huan luyen nhu binh thuong
model.fit(X_train, y_train, epochs=2)
```

- Keras sẽ truyền `y_true` và `y_pred` (Tensor) vào `huber_fn` tự động trong mỗi batch, tính sai số, rồi tự động truy vết lại để tính đạo hàm cập nhật ngược.

Lưu ý khi tải (load) mô hình chứa Custom Loss

- Cấu trúc Keras không lưu trữ mã nguồn Python của hàm `huber_fn`.
- **Vấn đề:** Khi dùng `keras.models.load_model()` ở script khác, Keras sẽ báo lỗi "Unknown loss function".

```
# Lưu như bình thường
model.save("my_model_with_a_custom_loss.h5")

# Khi load, bạn BẮT BUỘC phải truyền từ điển (dictionary)
# khai báo mapping từ chuỗi sang đối tượng Python
model = keras.models.load_model(
    "my_model_with_a_custom_loss.h5",
    custom_objects={"huber_fn": huber_fn}
)
```

Xây dựng Custom Loss bằng OOP (Subclassing) (1/2)

- Nếu hàm Loss yêu cầu siêu tham số (như `threshold=1.0` không cố định), sử dụng Hàm Python rất bất tiện.
- Giải pháp OOP: Kế thừa lớp `keras.losses.Loss`. Khai báo `get_config()` để tự động lưu tham số.

```
class HuberLoss(keras.losses.Loss):  
    def __init__(self, threshold=1.0, **kwargs):  
        self.threshold = threshold  
        super().__init__(**kwargs)  
  
    def call(self, y_true, y_pred):  
        error = y_true - y_pred  
        is_small_error = tf.abs(error) < self.threshold  
        squared_loss = tf.square(error) / 2  
        linear_loss = self.threshold * tf.abs(error) - self.threshold**2 / 2  
        return tf.where(is_small_error, squared_loss, linear_loss)
```

Xây dựng Custom Loss bằng OOP (Subclassing) (2/2)

- Cung cấp `get_config()` để lưu trữ tham số mạnh mẽ hơn.

```
def get_config(self):
    base_config = super().get_config()
    # Cập nhật thông số của mình kèm theo thông số cha
    return {**base_config, "threshold": self.threshold}

# Chỉ cần tạo đối tượng thay vì day string
model.compile(loss=HuberLoss(2.), optimizer="nadam")

# Keras gọi get_config để lưu threshold=2. vào JSON Model.
model.save("my_custom_model_v2.h5")

# Khi load, custom_objects vẫn là bắt buộc!
model = keras.models.load_model("my_custom_model_v2.h5",
    custom_objects={"HuberLoss": HuberLoss})
```

Tùy chỉnh Hàm Đánh giá (Custom Metrics)

- Bạn có thể tái sử dụng Custom Loss cho Metric. Loss dùng để huấn luyện, Metric in ra để đánh giá.

```
model.compile(loss="mse", optimizer="nadam",  
              metrics=[huber_fn])
```

- Vấn đề:** Metrics tiêu chuẩn Keras là "Streaming metrics", tính lũy kế qua các batch rồi chia trung bình. Hàm Python đơn giản sẽ bị tính sai lũy kế.
- Giải pháp nâng cao là kế thừa `keras.metrics.Metric`, quản lý các biến trạng thái bằng `self.add_weight()` và ghi đè hàm `update_state()`, `result()`.

Tùy chỉnh Lớp ẩn không trạng thái (Stateless Custom Layers)

- Nếu cấu trúc phi tuyến đặc biệt không chứa tham số cấu hình cần học (Weights/Biases), Keras cung cấp `keras.layers.Lambda`.

```
# Tạo tang mũ (exponential layer)
exponential_layer = keras.layers.Lambda(lambda x: tf.exp(x))

# Tích hợp vào Sequential Model
model = keras.models.Sequential([
    keras.layers.Dense(30, activation="relu"),
    keras.layers.Dense(1),
    exponential_layer
])
```

- Layer kiểu này có thể đóng vai trò làm activation function hoặc preprocessing.

Tùy chỉnh Lớp ẩn có trạng thái (Stateful Custom Layer) (1/2)

- Để tạo Layer riêng có khả năng "Học" hệ số W, b , bạn kế thừa `keras.layers.Layer`.
- Gồm 3 hàm: `__init__`, `build()` (khởi tạo Weights khi nhận dạng Shape) và `call()` (thực thi Feed-Forward).

```
class MyDense(keras.layers.Layer):
    def __init__(self, units, activation=None, **kwargs):
        super().__init__(**kwargs)
        self.units = units
        self.activation = keras.activations.get(activation)

    def build(self, batch_input_shape):
        self.kernel = self.add_weight(
            name="kernel", shape=[batch_input_shape[-1], self.units],
            initializer="glorot_normal")
```


Tùy chỉnh Lớp ẩn có trạng thái (Stateful Custom Layer) (2/2)

```
# Tiếp tục hàm build
self.bias = self.add_weight(
    name="bias", shape=[self.units], initializer="zeros")
super().build(batch_input_shape)

def call(self, X):
    return self.activation(X @ self.kernel + self.bias)

def compute_output_shape(self, batch_input_shape):
    return tf.TensorShape(batch_input_shape.as_list()[:-1] + [self.units])
```

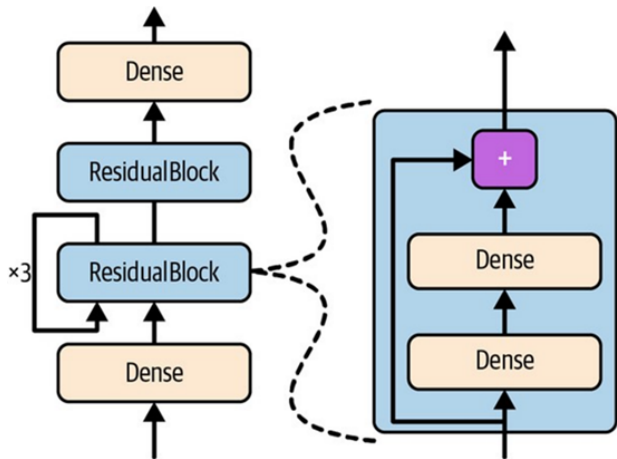
- Bằng cách này, Layer tùy biến có thể nối chuỗi trong Sequential một cách trong suốt.

Tùy chỉnh toàn bộ Mô hình (Custom Models)

- Khi xây dựng kiến trúc không dạng tuần tự (VD: mạng lưới có đường chéo, Residual Neural Networks), `Sequential` trở nên cồng kềnh.
- **Phương pháp Model Subclassing:** Cho phép ta thiết kế bằng Python tự do: Có thể dùng `if...else` luồng điều hướng, vòng lặp `for` để nối module động.
- Ta sẽ trình bày mô hình mẫu sử dụng một lớp khối (Residual Block) với khả năng *Skip Connection*.

Mô hình tùy chỉnh: Residual Block

- Hình 12-3 phác thảo cấu trúc một kiến trúc tùy ý với kỹ thuật **Skip connection**.
- Đường đi thẳng truyền trực tiếp Input cộng với Output của chuỗi tầng ẩn.
- Giải quyết triệt để lỗi mất gradient (Vanishing Gradient) trong Deep Learning.



Hình

12-3: Ví dụ mô hình tùy chỉnh: một mô hình tùy ý với lớp *ResidualBlock* tùy chỉnh chứa kết nối

Mã Python Xây dựng Residual Block

- Viết một Custom Layer đóng vai trò chứa chuỗi Dense và làm phép tính + kết nối.

```
class ResidualBlock(keras.layers.Layer):  
    def __init__(self, n_layers, n_neurons, **kwargs):  
        super().__init__(**kwargs)  
        self.hidden = [keras.layers.Dense(n_neurons, activation="elu")  
                        for _ in range(n_layers)]  
  
    def call(self, inputs):  
        Z = inputs  
        # Vòng lặp for feed-forward  
        for layer in self.hidden:  
            Z = layer(Z)  
        # Phep Cong Skip Connection  
        return inputs + Z
```

Xây dựng Custom Model sử dụng Residual Block

- Kế thừa `keras.Model` giống kế thừa `Layer`, cấu trúc `__init__` và `call()` tương tự.

```
class ResidualRegressor(keras.Model):
    def __init__(self, output_dim, **kwargs):
        super().__init__(**kwargs)
        self.hidden1 = keras.layers.Dense(30, activation="elu")
        self.block1 = ResidualBlock(2, 30)
        self.block2 = ResidualBlock(2, 30)
        self.out = keras.layers.Dense(output_dim)

    def call(self, inputs):
        Z = self.hidden1(inputs)
        for _ in range(1 + 3): # 4 lần đi qua Block 1
            Z = self.block1(Z)
        Z = self.block2(Z) # 1 lần đi qua Block 2
        return self.out(Z)
```

Model Subclassing vs Functional API

- **Subclassing API** cung cấp sự linh hoạt cực lớn (dùng vòng for, luồng logic động).
- Tuy nhiên, nhược điểm chí mạng là Keras **không thể lưu đồ thị** `model.summary()` hoặc vẽ `plot_model()` khi chưa truyền thử bộ dữ liệu (*Model is not Built*).
- Lỗi logic luồng xử lý bên trong Python rất khó phát hiện.
- **Lời khuyên:** Hãy dùng Functional API nếu có thể, trừ phi mạng có vòng lặp động hoặc điều kiện tùy thuộc dữ liệu sinh ra (dynamic branching).

Chế độ Tức thời (Eager) vs Chế độ Biểu đồ (Graph)

- TensorFlow 1.x yêu cầu định nghĩa đồ thị trước, rồi mở `tf.Session` để thực thi (Graph Execution). Quá trình debug vô cùng tàn bạo.
- TensorFlow 2.x đổi phương thức hoạt động mặc định thành **Eager Execution** - Chế độ thực thi tức thì.
- Mã chạy dòng nào có kết quả ngay. Gỡ lỗi bằng `print()` truyền thống.
- **Khuyết điểm:** Eager Mode Chậm hơn nhiều so với Graph (không thể tối ưu song song hóa phần cứng).

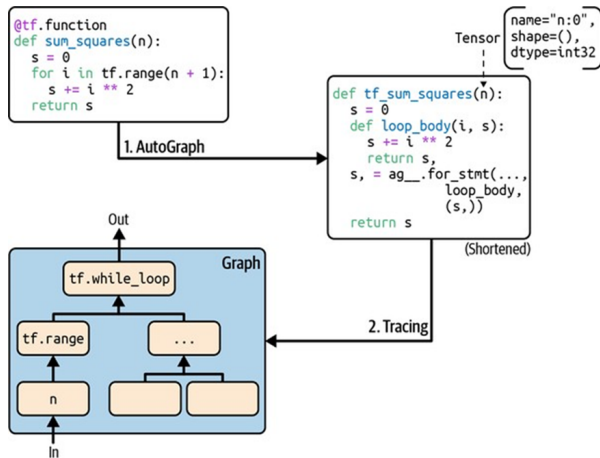
@tf.function: Phép màu của AutoGraph

- TensorFlow 2 mang đến giải pháp dung hòa bằng AutoGraph!
- Bạn viết code Python bình thường, chỉ cần đánh dấu @tf.function. TF sẽ tự động dịch thành đồ thị C++ ẩn bên dưới.

```
def cube(x):  
    return x ** 3  
  
# Tính toán - Nhanh cho Test nhưng chậm khi huấn luyện  
cube(tf.constant(2.0))  
  
# Khai báo TF Function bằng decorator  
@tf.function  
def tf_cube(x):  
    return x ** 3  
  
# tf_cube trở thành một computational graph!  
tf_cube(tf.constant(2.0))
```


Cách AutoGraph tạo Biểu đồ và Tracing (1/2)

- Hình 12-4 mô tả cơ chế của AutoGraph.
- Mã Python được quét, tạo Abstract Syntax Tree.
- Sinh ra hàm Python nâng cấp gọi các toán tử điều khiển lỗi của TF như `tf.cond`, `tf.while_loop`.



Hình

12-4: Cách TensorFlow tạo biểu đồ bằng AutoGraph và tracing

Cách AutoGraph tạo Biểu đồ và Tracing (2/2)

- Khi thực thi lần đầu tiên (**Tracing**), nó vạch ra lộ trình các phép toán thành **TensorFlow Graph**.
- Lần thực thi tiếp theo chạy Graph trực tiếp.
- Chú ý: TensorFlow tạo Poly-morphism (Đa hình đồ thị). Nếu gọi Input `int32`, nó tạo Graph 1. Nếu gọi `float32`, lưu thêm Graph 2.

Giới hạn và Quy tắc sống còn của TF Functions (1/2)

- **Side effects:** Hàm `print()` hoặc thao tác IO **chỉ chạy một lần duy nhất** ở giai đoạn Tracing, không nằm trong Đồ thị.

```
@tf.function
def my_func(x):
    print("Ham dang duoc trace:", x.dtype)
    return x + 1

my_func(tf.constant(1.0)) # In log, Tao Graph float32
my_func(tf.constant(2.0)) # KHONG in log, Tai su dung Graph float32
my_func(tf.constant(10))  # In log, Tao Graph int32
```

- Muốn in log động? Sử dụng `tf.print()`.

Giới hạn và Quy tắc sống còn của TF Functions (2/2)

- **Biến ngoài hàm / List Python:** Nếu dùng `list.append()`, list bị "đóng băng" ở trace. Hãy dùng `tf.TensorArray`.
- Tránh sinh số ngẫu nhiên bằng Numpy (`np.random.rand()`), hãy sử dụng `tf.random`.

```
@tf.function
def add_random_noise(x):
    # SAI LAM: Gia tri ngau nhien bi dong bang
    # noise = np.random.randn()

    # CHUAN XAC: Gia tri ngau nhien sinh ra moi khi goi Graph
    noise = tf.random.normal(shape=[])
    return x + noise
```

Vòng lặp huấn luyện tùy chỉnh

- Phương thức `model.fit()` tiện lợi, nhưng không phải kiến trúc nào cũng xài được (VD: mạng GAN huấn luyện Generator 1 bước, Discriminator 1 bước).
- Viết vòng lặp tùy chỉnh (Custom Training Loop) mất công, nhưng cho phép tùy biến không giới hạn.

tf.GradientTape(): Tính đạo hàm tự động

- TensorFlow cung cấp GradientTape đóng vai trò "băng ghi âm" lưu vết Tensor để lan truyền ngược (Backpropagation) tính đạo hàm tự động.

```
def f(w1, w2):  
    return 3 * w1 ** 2 + 2 * w1 * w2  
  
w1, w2 = tf.Variable(5.), tf.Variable(3.)  
  
with tf.GradientTape() as tape:  
    z = f(w1, w2)  
  
# Yêu cầu Tape tính đạo hàm Z theo biến w1, w2  
gradients = tape.gradient(z, [w1, w2])  
print(gradients)  
# Kết quả: dz/dw1=36, dz/dw2=10
```

GradientTape nâng cao

- Mặc định, 'GradientTape' chỉ theo dõi biến 'tf.Variable'. Không theo dõi 'tf.constant'.
- Khi gọi phương thức 'gradient()', bộ nhớ Tape bị xóa.
- Tính nhiều lần đạo hàm (VD: Hessian), dùng cờ 'persistent=True'.

```
with tf.GradientTape(persistent=True) as tape:
    z = f(w1, w2)

dz_dw1 = tape.gradient(z, w1)
dz_dw2 = tape.gradient(z, w2)    # Van OK

# Giai phong bo nho bang tay (bat buoc)
del tape
```

Viết Vòng lặp huấn luyện - Khởi tạo

- Tạo mô hình, cấu hình siêu tham số, Optimizer, Metric và Dữ liệu.

```
l2_reg = keras.regularizers.l2(0.05)
model = keras.models.Sequential([
    keras.layers.Dense(30, activation="elu", kernel_regularizer=l2_reg),
    keras.layers.Dense(1, kernel_regularizer=l2_reg)
])

batch_size = 32
train_set = tf.data.Dataset.from_tensor_slices((X_train, y_train))
train_set = train_set.shuffle(len(X_train)).batch(batch_size)

optimizer = keras.optimizers.Nadam(learning_rate=0.01)
loss_fn = keras.losses.mean_squared_error
mean_loss = keras.metrics.Mean()
```


Viết Vòng lặp huấn luyện - Thực thi (Epochs & Steps)

- Lặp qua Epochs, trong Epochs lặp qua DataBatches.

```
epochs = 5
for epoch in range(epochs):
    for step, (X_batch, y_batch) in enumerate(train_set):

        # 1. Forward pass ghi bang Tape
        with tf.GradientTape() as tape:
            y_pred = model(X_batch, training=True)
            main_loss = tf.reduce_mean(loss_fn(y_batch, y_pred))
            loss = tf.add_n([main_loss] + model.losses)

        # 2. Tính Gradient theo model variables
        gradients = tape.gradient(loss, model.trainable_variables)

        # 3. Kích hoạt phương thức cập nhật trọng số (Optimize)
        optimizer.apply_gradients(zip(gradients, model.trainable_variables))
```

Cấu trúc hoàn chỉnh và Những lưu ý quan trọng

- Bạn phải chịu trách nhiệm đảm bảo mọi thứ chính xác:
 - 1 Truyền cờ `training=True` vào lệnh dự đoán của model để *BatchNormalization/Dropout* hoạt động đúng.
 - 2 Theo dõi Metric thủ công (`update_state(loss)`), in ra cuối Epoch và gọi `reset_states()`.
 - 3 Xử lý cập nhật biến hệ thống (Non-trainable variables, như trung bình trượt của Batch Norm).

Tối ưu hóa Vòng lặp với @tf.function

- Vòng lặp for Python qua DataBatches siêu chậm.
- Lời khuyên: Đóng gói quá trình xử lý 1 Bước (1 Step) thành @tf.function để tăng tốc biên dịch toán học C++.

```
@tf.function
def train_step(X_batch, y_batch):
    with tf.GradientTape() as tape:
        y_pred = model(X_batch, training=True)
        main_loss = tf.reduce_mean(loss_fn(y_batch, y_pred))
        loss = tf.add_n([main_loss] + model.losses)
    gradients = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(gradients, model.trainable_variables))

# Trong vong lap chi gọi train_step(X_batch, y_batch)
```

- **Cốt lõi TensorFlow** đại lượng Tensors cực kỳ giống NumPy, tối ưu đa phần cứng (GPU/TPU).
- Việc **tùy biến Keras (Custom Component)** cho phép người học và nghiên cứu làm mới hoàn toàn Losses, Metrics, Layers, Models.
- **AutoGraph (@tf.function)** giải quyết sự tranh chấp linh hoạt kịch bản (Python) với biên dịch nhanh chóng.
- **tf.GradientTape** đưa quyền kiểm soát hoàn toàn tới luồng huấn luyện cho các nghiên cứu mới.